

Lab Report

Course Code: CSE-4742
Course Title: Computer Graphics Lab

Autumn 2026
Section: 7BF

Lab No: 1

Name of Labwork: Implementation of DDA Line Drawing Algorithm

Student's ID : C233456
Date of Performance : 06-09-2026
Date of Submission : 05-09-2026

Marks : _____

1. Number & Name of the Experiment

Lab 1: Implementation of DDA Line Drawing Algorithm

2. Introduction

The Digital Differential Analyzer (DDA) is an incremental line drawing algorithm used in computer graphics to generate a straight line between two given points. It determines the intermediate points of the line by using the differences between the starting and ending x and y coordinates.

First, the horizontal and vertical differences are calculated as $dx = x_2 - x_1$ and $dy = y_2 - y_1$. The number of steps is then selected as the larger of $|dx|$ and $|dy|$. This ensures that the line is drawn continuously without skipping pixels along the major direction. The incremental changes in the x and y coordinates are calculated by dividing dx and dy by the number of steps.

Starting from the initial point, these increments are repeatedly added to obtain the next position. Since the calculated coordinates may be fractional, they are rounded to the nearest integer pixel before being plotted on the raster screen. In this way, DDA converts the mathematical line into a sequence of pixels that can be displayed on the screen.

DDA is simple to understand and implement and can handle lines with different slopes and directions. However, it relies on floating-point calculations and rounding at each step, which can make it less efficient than integer-based line drawing algorithms such as Bresenham's algorithm.

3. DDA Line Drawing Algorithm

The Digital Differential Analyzer (DDA) is an incremental line drawing algorithm used to convert a mathematical line into pixels on a raster screen. The algorithm considers two cases depending on the magnitude of the changes in the x and y directions.

Let

$$\Delta x = x_2 - x_1$$

and

$$\Delta y = y_2 - y_1$$

The slope of the line is

$$m = \frac{\Delta y}{\Delta x}$$

Case 1: $|\Delta x| \geq |\Delta y|$

In this case, the change in the x direction is greater than or equal to the change in the y direction. Therefore, x is used as the driving coordinate.

For each step, the change in x can be taken as one pixel:

$$x_{i+1} = x_i + 1$$

Since

$$m = \frac{\Delta y}{\Delta x}$$

and $\Delta x = 1$, the corresponding change in y is

$$\Delta y = m\Delta x = m(1) = m$$

Therefore,

$$y_{i+1} = y_i + m$$

Hence, the next point is calculated as

$$\boxed{x_{i+1} = x_i + 1}$$

$$\boxed{y_{i+1} = y_i + m}$$

For a line drawn in the negative x direction, the sign is preserved, so the increment becomes -1 and the corresponding y increment also takes the appropriate sign.

Case 2: $|\Delta x| < |\Delta y|$

In this case, the change in the y direction is greater than the change in the x direction. Therefore, y is used as the driving coordinate.

For each step, the change in y can be taken as one pixel:

$$y_{i+1} = y_i + 1$$

From the slope equation,

$$m = \frac{\Delta y}{\Delta x}$$

we obtain

$$\Delta x = \frac{\Delta y}{m}$$

Since $\Delta y = 1$,

$$\Delta x = \frac{1}{m}$$

Therefore,

$$x_{i+1} = x_i + \frac{1}{m}$$

Hence, the next point is calculated as

$$\boxed{y_{i+1} = y_i + 1}$$

$$\boxed{x_{i+1} = x_i + \frac{1}{m}}$$

Again, the signs of the increments are adjusted according to the direction of the line.

Implementation

In the implementation, the two cases are handled using a single generalized loop. First, the differences and their absolute values are calculated:

$$dx = x_2 - x_1, \quad dy = y_2 - y_1$$

The number of steps is selected using the larger absolute difference:

$$\text{steps} = \max(|dx|, |dy|)$$

The increments used by the program are then calculated as

$$X_{\text{inc}} = \frac{dx}{\text{steps}}$$

and

$$Y_{\text{inc}} = \frac{dy}{\text{steps}}$$

Thus, when $|\Delta x| \geq |\Delta y|$, the horizontal increment has a magnitude of 1 and the vertical increment is based on the slope. When $|\Delta x| < |\Delta y|$, the vertical increment has a magnitude of 1 and the horizontal increment is based on the reciprocal of the slope.

The program starts from (x_1, y_1) and repeatedly adds these increments to obtain the next position. Since the calculated coordinates can be fractional, they are rounded to the

nearest integer pixel before being plotted.

The main steps of the implementation are:

1. Calculate dx and dy .
2. Find the absolute values $|dx|$ and $|dy|$.
3. Select the larger value as the number of steps.
4. Calculate the x and y increments.
5. Start from (x_1, y_1) .
6. Round the calculated coordinates to obtain integer pixel positions.
7. Plot each pixel using `putpixel()`.
8. Continue until the required number of steps is completed.

The `DDA()` function in the C++ program performs these calculations. The `std::abs()` function obtains the absolute differences, while the conditional expression selects the larger difference as the number of steps. The variables `xIncrement` and `yIncrement` store the incremental changes in the two directions.

Inside the loop, `std::round()` converts the floating-point coordinates into integer pixel coordinates, and `putpixel()` plots the pixel on the graphics window. The calculated pixel coordinates are also displayed using `cout`. The `delay(10)` statement provides a small delay so that the line-drawing process can be observed. If the starting and ending coordinates are identical, the program handles this special case by plotting a single pixel and terminating the function.

4. Program with Screenshot

The DDA line drawing algorithm was implemented in C++ using Visual Studio Code with the graphics library.

Code Screenshot

The following screenshot shows the complete implementation:

```

1  #include <math>
2  #include <iostream>
3  #include <conio.h>
4  using namespace std;
5  // $file="C233456_dda"; g++ "$file.cpp" -o "$file" -lbg -lgdi32 -lcomdlg32 -luuid -loleaut32 -lole32
6  void DDA(int x1, int y1, int x2, int y2)
7  {
8      int dx = x2 - x1;
9      int dy = y2 - y1;
10
11      int absDx = std::abs(dx);
12      int absDy = std::abs(dy);
13      int steps = (absDx > absDy) ? absDx : absDy;
14
15      if (steps == 0)
16      {
17          putpixel(x1, y1, WHITE);
18          return;
19      }
20
21      float xIncrement = (float)dx / steps;
22      float yIncrement = (float)dy / steps;
23
24      float x = x1;
25      float y = y1;
26
27      for (int i = 0; i <= steps; i++)
28      {
29          int pixelX = std::round(x);
30          int pixelY = std::round(y);
31
32          putpixel(pixelX, pixelY, WHITE);
33          cout << "(" << pixelX << ", " << pixelY << ")" << endl;
34
35          x += xIncrement;
36          y += yIncrement;
37
38          delay(10);
39      }
40
41  }
42  int main()
43  {
44      initwindow(800, 600, (char*)"DDA Line Drawing");
45      outtextxy(10, 10, (char*)"C233456");
46      int x1 = 400, y1 = 300;
47      int x2 = 100, y2 = 100;
48
49      DDA(x1, y1, x2, y2);
50      getch();
51      closegraph();
52      return 0;
53  }

```

Figure 1: DDA line drawing program

5. Output/Result

The program was successfully executed and produced a straight line between the specified starting and ending points.

The ID(C233456) was displayed in the graphics window as required.



Figure 2: Output of DDA line drawing algorithm from (100,100) to (400,300)

6. Discussion

The DDA (Digital Differential Analyzer) algorithm successfully generated a straight line between the given starting and ending coordinates. The algorithm determines the number of steps using the larger absolute difference between the changes in the x and y coordinates.

For each step, the algorithm calculates the incremental changes in x and y and uses floating-point values to determine the next pixel position. The calculated coordinates are then rounded and plotted on the screen to form the line.

The experiment demonstrates how a mathematical line can be represented as a sequence of pixels on a raster screen. It also shows how incremental calculations and coordinate rounding are used in computer graphics to perform line rasterization. Different input coordinates were used to observe the resulting line.

7. Conclusion

The DDA line drawing algorithm was successfully implemented in C++ using Visual Studio Code with the graphics library. The experiment provided an understanding of how a straight line can be generated pixel by pixel using incremental calculations.

The experiment also demonstrated the use of floating-point increments, rounding, and rasterization in computer graphics. Overall, the DDA algorithm provides a simple method for converting mathematical line coordinates into pixels for display on a raster screen.